



Acceptance of software process innovations – the case of extreme programming

George Mangalaraj¹,
RadhaKanta Mahapatra² and
Sridhar Nerur²

¹Department of Information Systems and
Decision Sciences, College of Business and
Technology, Western Illinois University, U.S.A.;

²Department of Information Systems and
Operations Management, College of Business
Administration, University of Texas at Arlington,
Arlington

Correspondence: George Mangalaraj,
Department of Information Systems and
Decision Sciences, College of Business and
Technology, Western Illinois University,
1 University Circle, Macomb,
IL 61455-1390, U.S.A.
Tel: (309) 298 1027;
Fax: (309) 298 1696;
E-mail: G-Mangalaraj@wiu.edu

Abstract

Extreme programming (XP), arguably the most popular agile development methodology, is increasingly finding favor among software developers. Its adoption and acceptance require significant changes in work habits inculcated by traditional approaches that emphasize planning, prediction, and control. Given the growing interest in XP, it is surprising that there is a paucity of research articles that examine the factors that facilitate or hinder its adoption and eventual acceptance. This study aims to fill this void. Using a case study approach, we provide insights into individual, team, technological, task, and environmental factors that expedite or impede the organization-wide acceptance of XP. In particular, we study widely differing patterns of adherence to XP practices within an organization, and tease out the various issues and challenges posed by the adoption of XP. Based on our findings, we evolve factors and discuss their implications on the acceptance of XP practices.

European Journal of Information Systems (2009) **18**, 344–354.

doi:10.1057/ejis.2009.23; published online 28 July 2009

Keywords: agile methodologies; extreme programming (XP); organizational adoption of XP

Introduction

Recent emergence of agile methodologies as a potential dominant design has significantly altered the dynamics of the software development environment. Agile development methodologies are challenging the established practice of plan-driven methodologies. Results of a recent survey attest to the increasing popularity and wide acceptance of these methodologies (Ambler, 2007).

Extreme programming (XP) is a relatively young methodology compared with traditional practices (e.g., waterfall approach) that have been dominant for several decades. XP practices are still evolving and vary from project to project. Therefore, there is a marked difference between 'XP in theory' and 'XP in use.' We now find that increasingly organizations perceive XP to be a viable methodology that can be adopted across all their projects. Our primary objective, therefore, is to understand and elucidate this growing phenomenon of organizational adoption of XP. We view XP and other agile methodologies as software process innovations (SPI) (Baskerville *et al.*, 2003). According to Kwon & Zmud (1987), diffusion of innovation is a multi-stage process, comprising initiation, adoption, adaptation, acceptance, use, and incorporation. Hitherto, academic research has concentrated primarily on the adoption of agile methods (e.g., Mahanti, 2006) and the adaptation of these methods to suit specific projects (e.g., Fitzgerald *et al.*, 2006). While adoption and adaptation studies throw light on the early phases of the diffusion of an innovation,

Received: 30 April 2008
Revised: 30 September 2008
2nd Revision: 20 June 2009
Accepted: 22 June 2009

acceptance and use studies are equally important to understand successful diffusion and subsequent incorporation within the organization.

Given the burgeoning interest in agile methods, an understanding of the factors that facilitate or hinder their acceptance and use in organizations would be invaluable to those considering adopting this new approach to software development. To this end, this study borrows from the rich body of knowledge available in SPI and diffusion of innovation areas to study the acceptance phase of agile software innovation. In particular, we used a case study research method to understand factors that influence the acceptance of XP among various project groups in an organization.

The contributions of this study to the SPI research are twofold. Firstly, a majority of the research on the diffusion of SPI has concentrated on issues related to the adoption stage, whereas this study focuses on the acceptance stage. Secondly, this study involves project teams as the units of analysis, which is a significant departure from prior studies that have used either an organizational or an individual context to study adoption of SPI. With the increasing importance of teams in the software development process, our knowledge of SPI will be incomplete without an understanding of how team characteristics influence the acceptance of SPI.

The organization of the paper is as follows. The next section provides the background literature for the study and identifies gaps in the research. This is followed by a discussion of the research methodology and the framework for SPI acceptance. The subsequent section discusses results of our analysis. Finally, we present the implications and conclude with directions for future research.

Agile software methods: gaps in the research

A growing frustration with the poor success rates of software projects has fueled the evolution of agile software development methodologies (Nerur *et al.*, 2005), which include XP, Scrum, Adaptive Software Development, Feature-Driven Development, Lean Development, and others. These methodologies reflect the principles articulated in the agile manifesto (Agile Alliance, 2001). They address the imperative for agility and flexibility in software development through their emphasis on people, on working software of high quality, on active customer involvement in the development process, and on accepting and creating change to create value (Cockburn, 2002; Highsmith, 2003). These developments have afforded researchers opportunities to further their understanding of the adoption and diffusion of SPI.

The diffusion of innovation is a multi-phased process that involves initiation, adoption, and implementation (Kozar, 1989). Kwon & Zmud (1987) divided the Information System (IS) implementation process into six phases, as shown in Figure 1. Initiation is the first step in the diffusion process. One feels the need to change from past practices. It is followed by adoption, which involves making resources, such as budget and people, available to implement the new practices. Adoption of a new software development process may require changes in the organizational structure, culture, and practices, as well as in individual behavior. It has been argued that the adoption of agile methodologies is likely to create significant burden on organizations (Nerur *et al.*, 2005). During the adaptation phase, adopters of a new software development methodology may modify the recommended practices to suit the specific needs of a project and/or an organization. The adapted process is then accepted by the adopters, and is assimilated and eventually institutionalized through frequent use. Academic research on SPI in general, and XP in particular, has focused predominantly on adoption/adaptation phases. This study focuses on the acceptance phase, which is deemed to be important in enriching our understanding of downstream phases of SPI (Karahanna *et al.*, 1999).

Adoption of SPI

SPI have attracted the attention of IS and management researchers for over two decades, resulting in a rich body of knowledge on salient factors that impact their adoption. Most of the research on SPI adoption has been carried out either at the organizational or the individual levels of analysis. Studies on organizational adoption of SPI have articulated various factors, such as environmental, organizational, and innovation characteristics, that influence adoption decisions (Leonard-Barton, 1987; Orlikowski, 1993; Cho & Kim, 2001). Individual-level studies, on the other hand, have used various theories such as the Theory of Reasoned Action and the Theory of Planned Behavior to explain the adoption behavior of individuals (Agarwal & Prasad, 2000; Riemenschneider & Hardgrave, 2001; Hardgrave & Johnson, 2003). Agile methodologies privilege people to rigid processes in order to achieve agility. With their advent, software development processes that were once predominantly technical now place greater emphasis on the social aspects of computing. The role of teams and the interaction among team members take center stage in driving successful agile projects. Therefore, there is a need to study SPI adoption behavior at the team level.

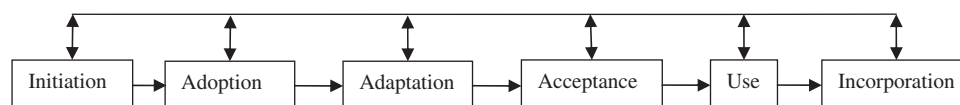


Figure 1 The innovation diffusion cycle (adapted from Kwon & Zmud (1987)).

Process adaptation/tailoring

As suggested by Kwon & Zmud (1987), adaptation follows adoption of an innovation. During the adaptation phase, the SPI is modified to suit the specific needs of the organization and/or the individual adopter. Successful adaptation is critical for the long-term success of SPI. Agile methodologies grew out of the need to adapt to changing requirements in a dynamic business environment. As organizations start experimenting with agile practices to supplant the hitherto practiced plan-driven software development methodologies, they find some practices easier to implement than others, contingent upon the context of implementation. A few research studies address this issue. Fitzgerald *et al.* (2006) examined tailoring of agile methods and found that practices in XP and Scrum complement each other and that organizations may selectively mix these methods to get the most benefit. Fruhling & De Vreede (2006) analyzed the implementation of XP in an organization and found that the adopters modified some of the XP practices such as pair-programming, testing, and on-site customer to suit their specific project needs. In their study of a large and complex project involving XP, Cao *et al.* (2004) highlighted the need to adapt some of the XP practices such as pair-programming, simple design, and customer engagement to suit unique project characteristics.

Conboy *et al.* (2007) examined the post-adoption stages of agile practices in three organizations and found that not only are the rates of assimilation different, but also the degree to which agile practices are assimilated depends on the specific needs of the teams. In summary, we have identified two limitations in the extant body of literature on SPI adoption. First, the post-adoption acceptance and performance of SPI have not been adequately researched (for example, see Jaspersen *et al.* (2005)). This viewpoint is underscored by Karahanna *et al.* (1999), who advocate the importance of studying post-adoption phases of IS innovations to improve our understanding of how to manage the process over time. The second limitation pertains to a lack of understanding of the role of teams in SPI adoption. Team members work on a common set of tasks and engage in behaviors acceptable to the group as a whole (Sarker *et al.*, 2005). Considering the increasingly important role of teams in agile software development, there is a need to understand the overall implementation process at the work group level. This view has been echoed by other IS researchers as well (Hardgrave *et al.*, 2003; Sarker *et al.*, 2005). Our study aims to address both these gaps in SPI adoption by studying the differing levels of acceptance of XP practices by teams within the same organization. Thus, it complements the work of Conboy *et al.* (2007) that examines different stages of assimilation of agile practices.

Research methodology

A case study research methodology was employed in this study because it enables the researcher to investigate a contemporary phenomenon within its natural setting

(Yin, 1994). Moreover, this methodology is ideally suited for our research objective of evolving a theory from the ground up (Eisenhardt, 1989) to offer insights into adoption of XP by an organization.

Site selection

We employed a single-site, multiple-case design for our research. Our unit of analysis was a project group. A single-site design ensures that factors extraneous to the organization, such as market forces, uniformly impact study variables across cases (Mustonen-Ollila & Lyytinen, 2003). This design also enabled us to control for variations in organizational culture and structure that may otherwise confound the findings. ABC Corp., a leading vendor of software solutions, provided an ideal site to investigate our research questions. It is a multi-billion-dollar multinational company with operations spread over nearly 50 countries. Senior management at ABC Corp. decided to adopt XP as the primary development methodology in 2001–2002. Following this decision, all developers in the company underwent an extensive XP immersion training program conducted by an outside consultant. Following the open workspace practice recommended in XP, ABC Corp. eliminated individual cubicles in favor of open computer labs. It also organized periodic XP user group meetings to promote a shared understanding of issues related to XP implementation.

As part of an internal assessment, ABC Corp. conducted periodic surveys to evaluate how well various project groups adhered to XP practices. Such surveys not only informed senior management about the level of adoption of XP practices, but also provided valuable data to assess the impact of XP practices on project outcomes. In 2005, almost 3 years following initial adoption of XP, the surveys revealed wide variation in the use of XP practices across projects. This singular characteristic offered an opportunity to study issues related to the post-adoption phase of the diffusion of XP in ABC Corp.

Case selection

We used the process of theoretical replication to select our cases. This process involves selecting cases that exhibit contrasting outcomes (Yin, 1994). The selection of cases proceeded as follows. The basic tenets of XP were originally expressed as 12 core practices (Beck, 2000) and subsequently enhanced to 13 primary practices and 11 corollary practices (Beck & Andres, 2005). ABC Corp. had developed its own XP methodology comprising 15 practices based on the original 12 (see Table 1). It conducted periodic surveys to assess how well projects complied with these practices. This data, obtained from 23 projects, provided the basis for case selection. We used an approximation of the Analytic Hierarchy Process (Shtub *et al.*, 2005) for this purpose. We first asked an XP expert (a senior architect and XP evangelist) in ABC Corp. to provide a pairwise comparison of the 15 XP practices using a scale of 0–9, where 0 indicated

Table 1 Core XP practices adopted by ABC Corp

<i>XP practice</i>	<i>Definition*</i>	<i>Relative importance</i>
Release planning	Part of the planning game (original XP practice) in which the customer specifies the requirements for the project and creates the release plan based on the developer's cost/time estimates.	0.139
Customer team	Customer is part of the development team. He/she determines requirements and performs acceptance tests.	0.126
Collective ownership	Members of the project team collectively own the code and any one can change any of the code at any time.	0.122
Pair programming	Pair programming involves two programmers working side by side at one computer, collaborating on the design, coding, and testing on a continuous basis.	0.109
Iteration planning	A part of the planning game that involves planning for the 2-week iterations.	0.100
Sustainable pace	Teams work at a sustainable pace, usually 40 h per week. Team members work overtime only when needed.	0.098
Refactoring	Ongoing redesign of software to improve its responsiveness to change.	0.049
Simple design	Simplest design is done for the functionality that has been defined and not in anticipation of future requirements.	0.046
Unit tests	Involves the development of the test module prior to writing the code.	0.040
Small releases	Software is released in small increments that address the most valuable business requirements.	0.040
Acceptance tests	Customer writes the acceptance tests. Testing is largely automated.	0.037
Continuous integration	Builds are done every few hours to lessen code integration problems.	0.028
Metaphor	It is the common vision of how the system works. The metaphor is intended to provide a broad view of the project's goal.	0.023
Open workspace	Developers work in an open lab-like environment with no cubicles.	0.022
Coding standards	Rules that everyone in the team must follow in writing code.	0.020

*These definitions are based on the following sources: Beck, 2000; Jeffries, 2001; Highsmith, 2002.

Table 2 Comparison of the two cases

<i>Characteristics</i>	<i>Project X</i>	<i>Project Y</i>
Application type	New project	Maintenance and enhancement
Language/technologies	Java	C, C++, Java, Motif
Project size	Approximately 30,000 lines of code.	Several hundred thousand lines, with more than 10-year-old code base.
Tools used	IntelliJ and CruiseControl	C++ editor, SlickEdit, and ClearCase
Team size	6 members	7 members
XP Score (Maximum 4.0)	2.64	1.58

'No Relationship' between the two practices and 9 indicated 'Absolute Importance' of one practice over the other. Using this information, we computed the relative importance index of each practice as listed in Table 1.

A project's XP adherence score was computed using the following formula:

$$\sum_{i=1}^{i=15} p_i w_i$$

where p_i and w_i are the project's score from the survey and the priority vector (i.e., relative importance index), respectively, for the i th practice. These scores were used to select two projects that represented the extremes with regard to their adherence to XP practices. We will call these Project X and Project Y. Project X scored high,

whereas Project Y scored low on the use of XP practices. Table 2 compares critical parameters about these two cases.

Protocol development and data collection

Following procedures outlined by Yin (1994), a framework inspired by Kwon & Zmud (1987) was developed to guide our data collection efforts. The framework identifies five factors that play a role in the implementation of a technological innovation: (1) Individual Factors, (2) Structural Factors, (3) Technological Factors, (4) Task-related factors, and (5) Environmental factors. These factors have been extensively used in studies investigating SPI adoption. For instance, Leonard-Barton (1987) found that innovation characteristics, organizational influences, and personal characteristics of potential users

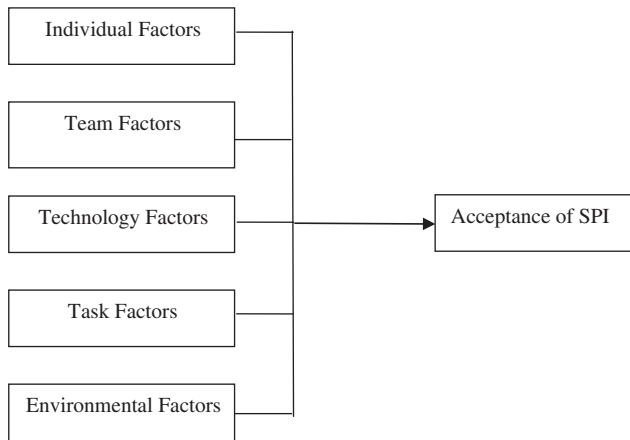


Figure 2 SPI acceptance frame work.

were salient factors in the acceptance of structured software development methodologies. Orlikowski (1993) used environmental, organizational, and IS context for studying the adoption of Computer Aided Software Engineering (CASE) tools. Mustonen-Ollila & Lyytinen (2003) used innovation, task, individual, environmental, and organizational factors in their longitudinal study on IS process innovations. Task-technology fit has also been found to play an important role in the adoption of SPI (Green *et al.*, 2005).

Since our study is based on a single organization, factors related to organizational structure and culture uniformly impact our cases. On the other hand, variations in team characteristics are likely to influence the adoption of XP. Therefore, we replaced structural factors proposed by Kwon & Zmud (1987) by team factors in the framework. The factors (see Figure 2) comprising our framework are described below.

Individual factors Although organizations mandate the use of a methodology, its universal acceptance and continued use really depend on the individuals who use it to develop software (Khalifa & Verner, 2000; Hardgrave & Johnson, 2003). Individual factors that impact acceptance of SPI include usability, usefulness (Hardgrave & Johnson, 2003), attitude, technical knowledge, tenure, and job insecurity (Agarwal & Prasad, 2000).

Team factors Software development is a collaborative endeavor. Characteristics of the project team are likely to influence the acceptance of an SPI. Hardgrave *et al.* (2003) suggest that social pressure may play a more critical role in the acceptance of a new methodology than do IT tools. Other salient factors in a group environment include opinions of high status individuals, substantive conflict within the group, and majority opinion in the group (Sarker *et al.*, 2005).

Technology factors Technology factors pertain to the technical characteristics of the innovation. These include compatibility, relative advantage, and complexity (Kwon & Zmud, 1987). Compatibility is the fit between the new SPI and the organizational and technological environment. Complexity in learning the new methodology also influences its level of acceptance (Hardgrave *et al.*, 2003). Relative advantage refers to additional benefits offered by the SPI vis-à-vis other innovations or the *status quo*. Tool support has been found to be important in the adoption of SPI (Green *et al.*, 2005).

Task factors Task factors are found to influence the adoption of software development methodologies. Fitzgerald (1998) claims that project size/scale and project type influence the use of such methodologies. Other factors, such as task type (e.g., maintenance vs new development), have been found to impact the adoption of Personal Software Processes (Green *et al.*, 2005).

Environmental factors Environmental factors also influence adoption of innovations. In her study on the adoption of CASE tools, Orlikowski (1993) considered environmental factors such as customers, competitors, and available technology. Kwon & Zmud (1987) regarded availability of and competition for resources as environmental factors. Such factors may also include project clients who influence the adoption/use of development methodologies (Fitzgerald, 1998).

The above framework guided the development of separate interview protocols for various roles performed by project group members. Specifically, interview protocols involving open-ended questions were developed for the following roles: developer, tester, architect, and development manager. Semi-structured face-to-face interviews were conducted to collect data. A total of 13 interviews, involving all members of both project groups, were carried out. In addition, follow-up interviews were conducted with the Chief Architect and the Chief XP Evangelist of the organization. The average duration of each interview was about 51 min. Interviews were tape recorded with the permission of the interviewees and were later transcribed.

Data analysis and findings

Data analysis proceeded in two steps: within-case analysis and across case analysis (Eisenhardt, 1989). Based on the SPI acceptance framework articulated earlier, we analyzed the data from each project separately and prepared a report on each. Subsequently, the two projects were contrasted with a view to explaining the differences in their levels of acceptance of XP practices. Table 3 summarizes the extent to which each project implements the advocated XP practices.

Individual factors

Software development is a knowledge intensive activity. Adoption of a new software development methodology

Table 3 Comparison of core XP practices across the two cases

XP Practice	Project X	Project Y
Release and iteration planning	Release planning is done periodically. Iteration planning involves all members of the team. Developers have the freedom to choose stories they wish to work on.	Release planning is done periodically. Although iteration planning involves all members of the team, stories are assigned based on skill specialization.
Customer team	Proxy customer writes user stories and collaborates in acceptance testing.	No dedicated customer/proxy customer is involved.
Collective ownership	Team members are collectively responsible for code development and maintenance.	Limited collective code ownership due to skill specialization.
Pair-programming	It is done to a greater extent. In general, team members seemed to enjoy pair-programming and they hold positive attitude toward it.	Pair programming was seldom done.
Sustainable pace	Team members determine deadlines and maintain a sustainable work pace.	Deadlines are determined by others (i.e., external customers) and sometimes entail work beyond normal hours to meet deadlines.
Refactoring	Refactoring is used frequently.	Large existing code base and resource constraints limited the use of refactoring.
Simple design	Tries to adhere to simple design concepts, but the actual design is primarily shaped by the architect.	Design is constrained by legacy code.
Small releases	Development proceeds in 2-week release planning cycles.	Development proceeds in 2-week release planning cycles. In addition, maintenance patches are released on a quarterly basis, and as per request of the customer.
Unit and acceptance tests	Test-driven development, including tracking of test coverage and automated testing, is practiced.	Test-driven development is not practiced.
Continuous integration	Builds are done daily using automated tools.	Builds are done less frequently, mainly upon request by team members.
Metaphor	Metaphor is not used.	Metaphor is not used.
Open workspace	Developers work in an open lab environment.	Developers use an open lab environment.
Coding standards	Coding standards are strictly enforced.	Coding standards are less rigorously enforced.

requires adopters to change their habits and follow the principles and practices recommended by the new methodology. A lack of knowledge about the new methodology can be a significant hindrance to its successful adoption and acceptance. Adopters are likely to fall back on their old work habits in the absence of adequate knowledge about the new methodology.

XP knowledge All development teams at ABC Corp. had participated in an XP immersion workshop organized by a leading consultant prior to the adoption of XP. In spite of this, we found significant differences between knowledge levels of members of the two project groups regarding XP principles and practices. Members of project X, in general, were highly knowledgeable about XP practices. According to one project X member, 'I have read several books, Kent Beck series, a number of book reviews, and participated in local discussion groups.' Another project X member had presented a paper at a premier XP conference. In contrast, some members of project Y exhibited a lack of understanding of some core XP practices. In response to our question regarding collective code ownership, a core XP practice, one project Y member responded, 'What is collective code

ownership?' With regard to practicing pair programming, another developer in project Y said, 'I did ask some questions to Jeff (name changed to maintain privacy) and I don't know whether that is pairing. He sat with me for a little while' These observations suggest that a lack of knowledge about XP practices would hinder their acceptance and use. This is in line with a recent study that found the need for continued XP training even after adoption (Fruhling *et al.*, 2008).

Attitude toward XP practices Adoption of XP was mandated by top management at ABC Corp. Team members did not have a choice to opt out. In spite of using XP for an extended period of time, there were differences among team members with regard to their attitude toward XP. Members of project X showed a positive attitude toward XP practices. This is evident by the statement of a developer, 'By and large most people are OK with it (*XP practices*). Some of them like it, some of them tolerate it. We have gotten rid of anybody who was on the other side of it.' Another member summed up his colleagues' attitude toward XP as 'I have to say based on our experience, we are better with XP than without XP.' In contrast, members of project Y had a negative

disposition toward XP, and pointed out various constraints (e.g., budgetary, scope, and tool) that made it difficult for them to follow XP practices. This is illustrated by the following comment of a developer on practicing pair programming, 'There is actually no need for it (pair programming). We never really did it. Because we need to practice XP sometimes we paired and did tasks.' Overall, this group showed less enthusiasm for XP. Therefore, it appears that a positive attitude toward XP practices will facilitate their acceptance by the project team.

Team factors

Teamwork plays a key role in the success of XP practices. It empowers developers by giving them key decision-making responsibilities, such as deciding the project scope and schedule. The project manager's responsibilities change from monitoring and managing to coaching and leading.

Team management Changing management practices from a command-and-control mode to one that empowers the team members and gives them more autonomy will facilitate acceptance of XP (Nerur *et al.*, 2005). Members of project X enjoyed considerable autonomy in the choice of tasks and in determining the schedule. This autonomy also required that they be familiar with the entire application and its goals. As stated by a project X member, 'I think in this project, everybody is on the same step. We try to make sure everybody understands.' Project Y, on the other hand, relied on specialized skills of members. Since overall knowledge about the project was not widely dispersed among team members, tasks were assigned based on a person's expertise. Further, members of project Y were unwilling to participate in overall project-related decisions. According to a project Y member, '(In XP) during the iteration ... anybody can pick-up any task (story) and work on it, but that is not how we do it.' According to another member, 'Because of the lack of expertise and across expertise (on all modules) is completely not there, we pretty much know who will do what task' For the same reason, collective code ownership was not practiced in this project group. These findings suggest that practices that empower team members will facilitate acceptance of XP.

Team leadership Development manager and coach in project X made significant efforts to ensure that XP practices were strictly followed by team members. According to a member, 'coach is more forceful and he tries to strictly enforce XP practices in the project.' Mentoring provided by the coach of project X was critical in getting the team members to accept XP practices. Leaders of project Y, on the other hand, played a more traditional project management role when compared to what is envisaged in XP. Unlike project X, which had a dedicated XP coach, the same person played the roles of development manager and XP coach in project Y. This duality of roles may be a plausible reason for the lack of

effort to ensure adherence to all XP practices in project Y. For instance, one member stated, 'There is definitely coaching and monitoring involved but not on all practices.' Further, much like a traditional project, all communications between the team and the customer were handled through the project manager. Thus, the XP coach plays a key role in ensuring adherence to XP practices by team members. This influential role of the XP coach has also been highlighted in the literature (e.g., Salo, 2004; Beck & Andres, 2005).

Technology factors

Implementation of a software development process greatly depends on the tools and technologies used. XP practices such as refactoring, test-driven development, and continuous integration are difficult to implement in the absence of appropriate tools. Similarly, the concepts of simple design and iterative development are easier to implement using an object-oriented programming language such as Java. Compatibility of the technology with XP practices and availability of automated tools were found to impact acceptance of XP practices.

Compatibility Project X was implemented using Java. On the other hand, project Y, which involved maintenance and enhancement support for a large legacy application, used multiple languages, including C, C++, Java, and Motif. The comments of the members of project Y suggested that the use of several languages in a legacy environment presents some serious challenges to the adoption of XP practices such as code ownership, refactoring, and automated testing. According to a member, 'If you are working on an old MOTIF or C++ code and you want to put an XP methodology on top of it, it is harder to get it accomplished.' Yet another developer said, 'Simple design is nice in paper but it does not always work. To practice XP and fit it with existing code is difficult.'

Tools support Software tools used in project X, such as IntelliJ and CruiseControl, have built-in support for some of the core XP practices such as refactoring, continuous integration, and test-driven development. These tools greatly helped members of project X in adhering to XP practices. This notion was echoed by several members of project X. According to one member, 'IntelliJ is a big factor in being successful. It greatly helps in refactoring the code quickly and safely.' Another member said, 'Tool support is definitely important and it varies according to the language. Without IntelliJ-like tool it may be hard to do refactoring.' In contrast, the tools and Integrated Development Environments employed in project Y offered little or no support for core XP practices. Because of this, members of project Y faced great difficulty in implementing some of the practices and had to improvise. According to a team member, 'Test first in C++ platform is difficult to implement; for Java they have IntelliJ and here we tried it but we could not.' Our

analysis of these two projects suggests that compatibility of technology with XP practices and availability of appropriate tools can facilitate the acceptance of XP.

Task factors

Characteristics of the development task can impact the level of acceptance of a new methodology. Application type and size of the project were found to influence the levels of acceptance of XP in the two projects. Project X involved the development of a new application to be marketed to prospective customers. It took about 16 months to complete and included 30,000 lines of code. There was no need to integrate project X with legacy applications. In contrast, project Y involved maintenance of and enhancement to a legacy code base made up of several hundred thousand lines of code developed over a 10-year period. Tight integration with legacy code made refactoring and automated testing extremely difficult to implement. Release cycle times were very short for the application with developers being asked to deliver patches/enhancements at periodic intervals. In contrast, the relatively small size and greenfield nature of project X made it easier for team members to implement several XP practices.

Environmental factors

The constraints imposed by the environment may impact the extent to which project teams accept XP practices. Budget and schedule constraints as well as customer participation were found to have significant influence on acceptance of XP practices.

Budget and schedule constraint Project X was an in-house development of a new product with relatively less restriction on budget and delivery schedule. This unique nature of the project afforded a lot of flexibility to the members of the team. As one of the members put it, 'There is the budget and there is timeline but we managed the scope so we are ready when we said we will be ready.' According to another member, 'We try to implement XP correctly. If you look at other teams, many of them have done less than what we have done towards the implementation of acceptance testing. Because it is hard and we had the time to do it.'

In contrast, project Y had strict budgetary constraints, and its scope and schedule were dictated by demanding product managers. Practices such as pairing were, therefore, viewed as wasteful due to additional demand on man-hours. Refactoring was not done in project Y because of the same reason. As one project Y member explained, 'We don't really have enough money to do complete refactoring exercise.' A tight schedule constraint imposed additional burden on the project team. A developer summed it up as follows, 'In the beginning we did little more of XP. So we tried to pair as much as we can even with time constraints we tried to practice XP. Nowadays we do it less. We have more time constraints.' Therefore, the acceptance of XP is facilitated when the

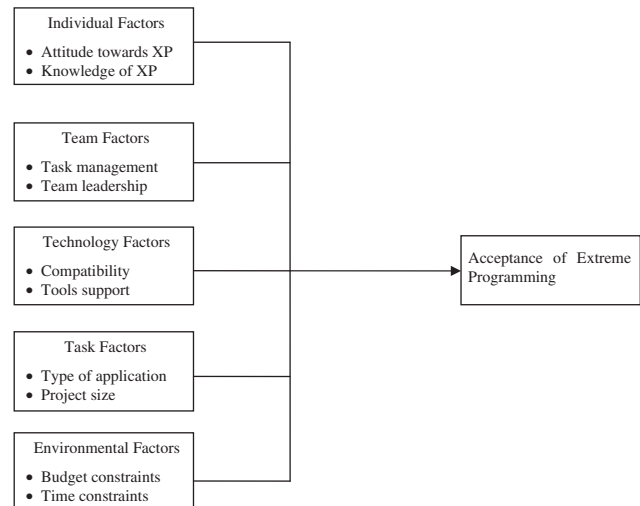


Figure 3 Extreme programming acceptance framework.

project team has greater control over its budget and schedule.

Figure 3 summarizes our findings. It is apparent that the factors included in the SPI framework offer insights into the acceptance of XP practices by organizations.

Results of this study were presented to senior members of ABC Corp., who found them to be meaningful in their efforts to facilitate organization-wide acceptance of XP practices. Subsequent discussions with them revealed two key issues: (a) members' attitude toward teamwork and (b) the role of proxy customers. XP, unlike traditional development approaches, relies extensively on team work. The adoption of XP was mandated by top management at ABC Corp. This mandate created strong resentment among many developers because of their reluctance to work in collaborating teams that used open workspaces. These developers considered it an encroachment on their personal workspace as well as on their style of working. Consequently, many of them left the organization. Perhaps, a strong objection to teamwork did not emerge from our interviews because of this early turnover. However, in a different setting, the attitude toward teamwork can play a pivotal role in the acceptance of XP practices. Likewise, the role of proxy customers can affect the acceptance of XP practices. As is evident in the literature, XP requires the active participation of customers in the development process. However, this is difficult to accomplish in practice. This came out clearly in our discussions with senior members of ABC Corp. Project X, by virtue of being an internal project, had the advantage of a proxy customer whose involvement favorably affected XP practices. On the other hand, project Y, which served the interests of several external customers, could not get their adequate participation. This unwillingness of external customers to commit time, money, and other resources to IS projects is not uncommon.

Implications of the study and future research

The findings of this study have important implications for practitioners and academics alike. From a practical perspective, this research elucidates how various factors either facilitate or impede the acceptance of XP principles. The study of two contrasting projects of similar team compositions within the same organization suggests that some characteristics may be conducive to the use of XP, while others may inhibit their acceptance. Our findings resonate with the general observation that agile methods are suitable for new projects of small scope. Since many organizations are planning to adopt agile development, quite possibly XP, the findings of this study will enable them to prepare better for the initiation, adoption, and adaptation of XP.

At the individual level, constant reinforcement of XP practices is required to ensure that developers understand the importance and value of the core principles. Suitable reward structures have to be established to encourage them to follow these principles. At the team level, greater adherence to XP practices may be achieved by ensuring that team members have autonomy in choosing tasks and by empowering them to make local decisions related to scope and estimates of duration. Team members should be provided incentives to choose tasks that are not in their area of expertise. As Ambler (2004) points out, team members should evolve into 'generalizing specialists.' The involvement of customers or their proxies as well as extensive communication among developers, customers/proxies, and testers is essential for successful acceptance.

XP may be more difficult to use in some projects than in others. In particular, when project teams are new to XP practices, the availability of appropriate tools and technologies will greatly facilitate the acceptance of XP. This study reinforces Boehm & Turner's (2004) observation that application type and size, among other things, are important considerations in the choice of a methodology. New and small applications are more conducive to the use of XP than large applications that involve maintenance and evolution of code. The problem is further exacerbated when integration with a large legacy code base is required. Strict budgetary and time constraints can be barriers to sustained use of XP. For example, pair-programming promises better quality of code, but requires additional man-hours to be expended. This can make adherence to XP practices problematic in situations involving tighter budgetary constraints.

About the authors

George Mangalaraj received his M.S. and Ph.D. degrees in Information Systems from the University of Texas at Arlington. Currently, he is an assistant professor of Information Systems at Western Illinois University,

For the research community, this research adapts and extends the innovation acceptance framework proposed by Kwon & Zmud (1987) in the context of agile software development. Further, ours is one of the few early studies that examine the post-adoption acceptance of a new software development methodology. Understanding the acceptance of SPI in an organizational context is important in order to better manage the diffusion of SPI. To this end, this study offers a set of factors that may be influential in the acceptance of XP in an organization.

While Conboy *et al.* (2007) studied the assimilation of agile practices in teams across multiple organizations, our research focuses on the variations in acceptance of XP practices across teams within the same organization. Thus, the two studies complement each other and provide valuable insights into this phenomenon. Clearly, more empirical research is required to gain a deeper understanding of the issues in the acceptance of agile practices. Further, there is an urgent need to evolve theoretically grounded models that can be empirically validated. We see our research as a step in this direction.

Conclusions

Recent reports suggest that the agile wave is in the ascendant (Ambler, 2007). Organizations that are seriously considering riding this wave have to be cognizant of the myriad issues and challenges that have to be overcome to initiate, adopt, adapt, and sustain this software innovation. While research efforts have been made to elucidate the impediments to agile adoption (Nerur *et al.*, 2005), there is very little empirical work on the critical success factors of agile adoption, and much less on the factors that would allow agile as an innovation to either thrive or flounder after its initial adoption. This study found individual, team, technological, task, and environmental factors to influence the acceptance of XP practices in an organization. Such studies that are geared toward elucidating issues in managing the change to new and conceptually different software development approaches are not only invaluable to practitioners, but also further our understanding of the theoretical foundations of the change process underlying SPI.

Acknowledgements

The authors thank the editorial board of the special issue and the two anonymous reviewers for their constructive feedback and helpful suggestions during the review of this manuscript.

Macomb. His research interests are in the areas of systems development, diffusion of innovations, and issues in online environment. His research publications appear in *Communications of the ACM*, *IEEE Transactions of*

Professional Communication, and various conference proceedings such as the America's Conference on Information Systems, the Hawaii International Conference on System Sciences, and the Decision Sciences Institute.

RadhaKanta Mahapatra is an associate professor of Information Systems at the University of Texas at Arlington. He holds a Ph.D. in Information Systems from Texas A&M University. His research interests include software development methodologies, knowledge management, data mining, web-based end-user training, and IT management. His research publications appear in journals such as *MIS Quarterly*, *Communications of the ACM*, *Decision Support Systems*, *Information & Management*, *Database for Advances in Information Systems*, *Communications of the AIS*, *Journal of Database Management*, *Journal of Computer Information Systems*, and *International Journal of Production Research*. He received the Distinguished

Research Publication Award and the Distinguished Professional Publication Award from the College of Business Administration of the University of Texas at Arlington.

Sridhar Nerur is an associate professor of Information Systems at the University of Texas at Arlington. He received his Ph.D. from the University of Texas at Arlington. His research interests include software development, citation analysis, reuse, maintenance, and philosophical aspects of systems development. He has published in the *MIS Quarterly*, *Strategic Management Journal*, *Communications of the ACM*, *Database for Advances in Information Systems*, *Communications of the AIS*, *Information Systems Management*, *Information Management & Computer Security*, and in various conference proceedings. He served as an associate editor of the *European Journal of Information Systems*.

References

- AGARWAL R and PRASAD J (2000) A field study of the adoption of software process innovations by information systems professionals. *IEEE Transactions on Engineering Management* **47**(3), 295.
- AGILE ALLIANCE (2001) Agile manifesto. [WWW document] <http://agilemanifesto.org> (accessed 26 February 2009).
- AMBLER S (2007) Survey says ... Agile has crossed the chasm. *Dr. Dobbs's Journal* **32**(8) [WWW document] <http://www.ddj.com/architect/200001986> (accessed 7 July 2009).
- AMBLER SW (2004) *The Object Primer: Agile Modeling-Driven Development With UML 2.0*. Cambridge University Press, Cambridge, U.K., New York.
- BASKERVILLE R, RAMESH B, LEVINE L, PRIES-HEJE J and SLAUGHTER S (2003) Is 'internet-speed' software development different? *IEEE Software* **20**(6), 70–77.
- BECK K (2000) *Extreme Programming Explained: Embrace Change*. Addison-Wesley, Reading, MA.
- BECK K and ANDRES C (2005) *Extreme Programming Explained: Embrace Change*. Addison-Wesley, Boston, MA.
- BOEHM BW and TURNER R (2004) *Balancing Agility and Discipline: A Guide for the Perplexed*. Addison-Wesley, Boston, MA.
- CAO L, MOHAN K, XU P and RAMESH B (2004) How extreme does extreme programming have to be? Adapting XP practices to large-scale projects. In *Proceedings of the 37th Hawaii International Conference on System Sciences* (SPRAGUE, RH, Ed.), IEEE, Hawaii.
- CHO H and KIM YG (2001) Critical factors for assimilation of object-oriented programming languages. *Journal of Management Information Systems* **18**(3), 125–156.
- COCKBURN A (2002) *Agile Software Development*. Addison-Wesley, Boston, MA.
- CONBOY K, PIKKARAINEN M and WANG X (2007) Agile practices in use from an innovation assimilation perspective: a multiple case study. In *Twenty Eighth International Conference on Information Systems*. (RIVARD S and WEBSTER J, Eds.), Montreal, Canada.
- EISENHARDT KM (1989) Building theories from case-study research. *Academy of Management Review* **14**(4), 532–550.
- FITZGERALD B (1998) An empirical investigation into the adoption of systems development methodologies. *Information & Management* **34**(6), 317–328.
- FITZGERALD B, HARTNETT G and CONBOY K (2006) Customising agile methods to software practices at intel shannon. *European Journal of Information Systems* **15**(2), 200–213.
- FRUHLING A and DE VREEDE G-J (2006) Field experiences with extreme programming: developing an emergency response system. *Journal of Management Information Systems* **22**(4), 39–68.
- FRUHLING A, MCDONALD P and DUNBAR C (2008) A case study: introducing extreme programming in a US government system development project. In *Proceedings of the 41st Hawaii International Conference on System Sciences*. (SPRAGUE RH, Ed.), IEEE, Hawaii.
- GREEN GC, HEVNER AR and COLLINS RW (2005) The impacts of quality and productivity perceptions on the use of software process improvement innovations. *Information & Software Technology* **47**(8), 543–553.
- HARDGRAVE B, DAVIS F and RIEMENSCHNEIDER C (2003) Investigating determinants of software developers' intentions to follow methodologies. *Journal of Management Information Systems* **20**(1), 123–151.
- HARDGRAVE B and JOHNSON R (2003) Toward an information systems development acceptance model: the case of object-oriented systems development. *IEEE Transactions on Engineering Management* **50**(3), 322–336.
- HIGHSMITH J (2003) Agile project management: principles and tools. Cutter Consortium Reports, Cutter Consortium, Arlington, MA.
- HIGHSMITH JA (2002) *Agile Software Development Ecosystems*. Addison-Wesley, Boston.
- JASPERSON JS, CARTER PE and ZMUD RW (2005) A comprehensive conceptualization of post-adoptive behaviors associated with information technology enabled work systems. *MIS Quarterly* **29**(3), 525–557.
- JEFFRIES R (2001) What is extreme programming? [WWW document] <http://www.xprogramming.com/xpmag/whatisxp.htm> (accessed 26 February 2009).
- KARAHANNA E, STRAUB DW and CHERVANY NL (1999) Information technology adoption across time: a cross-sectional comparison of pre-adoption and post-adoption beliefs. *MIS Quarterly* **23**(2), 183–213.
- KHALIFA M and VERNER JM (2000) Drivers for software development method usage. *IEEE Transactions on Engineering Management* **47**(3), 360–369.
- KOZAR KA (1989) Adopting systems development methods: an exploratory study. *Journal of Management Information Systems* **5**(4), 73.
- KWON TH and ZMUD RW (1987) Unifying the fragmented models of information systems implementation. In *Critical Issues in Information Systems Research* (BOLAND RJ and HIRSCHHEIM RA, Eds), pp 227–251, John Wiley & Sons Ltd., New York, NY.
- LEONARD-BARTON D (1987) Implementing structured software methodologies: a case of innovation in process technology. *Interfaces* **17**(3), 6–17.
- MAHANTI A (2006) Challenges in enterprise adoption of agile methods – a survey. *Journal of Computing and Information Technology* **14**(3), 197–206.

- MUSTONEN-OLLILA E and LYYTINEN K (2003) Why organizations adopt information system process innovations: a longitudinal study using diffusion of innovation theory. *Information Systems Journal* **13**(3), 275–298.
- NERUR S, MAHAPATRA R and MANGALARAJ G (2005) Challenges of emigrating to agile methodologies. *Communications of the ACM* **48**(5), 72–78.
- ORLIKOWSKI WJ (1993) Case tools as organizational change: investigating incremental and radical changes in systems development. *MIS Quarterly* **17**(3), 309–340.
- RIEMENSCHNEIDER CK and HARDGRAVE BC (2001) Explaining software development tool use with the technology acceptance model. *Journal of Computer Information Systems* **41**(4), 1–8.
- SALO O (2004) Improving software process in agile software development projects: results from two XP case studies. In *Proceedings of the 30th EUROMICRO Conference*. (STEINMETZ R and MAUTHE A, Eds.), IEEE, Rennes, France.
- SARKER S, VALACICH J and SARKER S (2005) Technology adoption by groups: a valance perspective. *Journal of Association for Information Systems* **6**(2), 37–71.
- SHTUB A, BARD JF and GLOBERSON S (2005) *Project Management: Processes, Methodologies, and Economics*. Pearson Prentice Hall, Upper Saddle River, NJ.
- YIN RK (1994) *Case Study Research: Design and Methods*. Sage Publications, Thousand Oaks, CA.